



JavaScript Hoisting Cheat Sheet



What is Hoisting?

Hoisting is a JavaScript mechanism where variable and function declarations are moved to the top of their scope before code execution. Only **declarations** are hoisted, not **initializations**.



Variable Hoisting

Declaration	Hoisted	Initialized	TDZ	Scope
var	Yes	Yes (undefined)	No	Function
let	Yes	No	Yes	Block
const	Yes	No	Yes	Block



Temporal Dead Zone (TDZ)

The TDZ is the phase between hoisting and declaration where `let` and `const` exist but are uninitialized. Accessing them throws a `ReferenceError`.

```
console.log(x); // ReferenceError
let x = 10;
```



Function Hoisting

- **Function Declarations** are hoisted with their full definition.
- **Function Expressions** and **Arrow Functions** are not fully hoisted.

```
sayHi(); // "Hi"
function sayHi() {
  console.log("Hi");
}
```

```
greet(); // TypeError: greet is not a function
var greet = function() {
  console.log("Hello");
}
```

+ Arrow Function Hoisting

Arrow functions behave like function expressions in hoisting.

```
arrow(); // TypeError: arrow is not a function
var arrow = () => {
  console.log("Arrow!");
}
```

With `let` or `const`, accessing before declaration throws `ReferenceError` due to TDZ.

```
arrow(); // ReferenceError
const arrow = () => {
  console.log("Hi");
}
```

IIFE (Immediately Invoked Function Expression) Hoisting

IIFEs are function expressions, not declarations — they are not hoisted. Calling them before their definition causes a `ReferenceError`.

```
myFunc(); // ReferenceError

(function myFunc() {
  console.log("Hello IIFE!");
})();
```

Interview Questions & Answers

Q1: What is hoisting?

Hoisting is JS's behavior of moving declarations to the top of the scope.

Q2: Does hoisting move initializations?

No. Only declarations are moved, not the assignments.

Q3: What is the Temporal Dead Zone?

It's the period where a `let` or `const` variable is hoisted but not yet initialized. Accessing it causes a `ReferenceError`.

Q4: Are function declarations hoisted?

Yes, with their full implementation.

Q5: Are arrow functions hoisted?

No. They are treated like function expressions. If declared with `var`, accessing before assignment gives `TypeError`. With `let` or `const`, it's `ReferenceError`.

Q6: Are IIFEs hoisted?

No. IIFEs are invoked expressions and are not hoisted.

Q7: How does hoisting differ for `var`, `let`, and `const`?

- **var** is hoisted and initialized to `undefined`.
- **let** and **const** are hoisted but not initialized (TDZ).

Q8: Can hoisting lead to bugs?

Yes, especially with `var`. Use `let` or `const` to avoid unexpected behavior.